SYSADMEOW

РУКОВОДСТВО ПО АРХИТЕКТУРЕ, НАСТРОЙКЕ И ЭКСПЛУАТАЦИИ

Учебный Android-маркетплейс разовых услуг удалённой настройки сетевого ПО

Параметр	Значение
Назначение	Техническая документация
Статус	MVP
Платформа	Android 24+
Архитектура	MVVM + StateFlow + Repository
Backend	Firebase Authentication + Cloud Firestore
Платёж	beGateway Android SDK 2.6.0
Язык	Kotlin
UI	Классические Android Views, XML, RecyclerView, CardView

© Фёдоров Владислав Владимирович

2026

Все права защищены

ПРЕДИСЛОВИЕ

ПРЕДИСЛОВИЕ

Как использовать этот документ для защиты, сопровождения и воспроизведения проекта

0.1 Назначение документа

Этот документ объединяет два привычно раздельных жанра: учебный отчёт и руководство разработчика. Поэтому читатель получает не только описание того, что реализовано, но и причинно-следственную картину проекта: какие роли существуют, где хранится состояние, как Android-слои взаимодействуют с Firebase, почему отзыв обновляет рейтинг транзакционно и в какой точке платёж становится заказом.

Документ описывает исключительно финальную версию SysAdMeow. История правок, миграции и промежуточные варианты реализации намеренно исключены. Это позволяет воспринимать документ как описание законченной системы, а не как дневник разработки.

0.2 Для кого предназначена книга

- для преподавателя — как структурированный отчёт с архитектурой, сценариями, схемой данных, интерфейсом и тестированием;
- для разработчика — как руководство по устройству модулей, репозиториев и Firebase-структуры;
- для студента — как пошаговая карта воспроизведения проекта с чистого Android-проекта;
- для сопровождения — как справочник по Security Rules, платёжному сценарию и ограничениям учебной версии.

> **IMPORTANT Граница между отчётом и эксплуатационной документацией**
>
> В документе намеренно объясняется не только «что сделано», но и «как это работает». Однако платёжная интеграция описывается в рамках учебной схемы; production-операции, требующие доверенного backend, вынесены в отдельный раздел.

0.3 Соглашения документа

Обозначение	Назначение
NOTE	Контекст, пояснение или практическая подсказка.
IMPORTANT	Решение, от которого зависит корректность или безопасность системы.
CAUTION	Ограничение учебной версии или потенциально опасный для production момент.
TIP	Короткая практическая рекомендация по работе с проектом.

Имена классов, пакетов, коллекций Firestore и полей приводятся в моноширинном написании, чтобы их можно было сопоставить с кодом проекта. Названия ролей CLIENT и ADMIN являются внутренними значениями; в пользовательском интерфейсе ADMIN называется «sysadmin-кот».

0.4 Карта документа

Основное повествование идёт от внешней модели продукта к внутренней реализации: сначала предметная область и роли, затем архитектура и данные, после этого — отдельные пользовательские подсистемы, безопасность и тестирование. Приложения содержат конфигурации и правила, необходимые для воспроизведения финальной сборки.

Содержание

01 Обзор системы и предметная область
02 Требования, роли и пользовательские сценарии
03 Технологический стек и структура проекта
04 Архитектура MVVM и управление состоянием
05 Firebase Authentication и постоянная сессия
06 Регистрация и кото-аватары
07 Cloud Firestore и модель данных
08 Каталог, категории и главный экран
09 Карточка специалиста и отзывы
10 Пагинация и агрегированный рейтинг
11 Корзина и защита от дублей
12 Платёжный модуль beGateway и checkout
13 Личный кабинет и публикация карточки
14 Системные уведомления
15 Firestore Security Rules
16 Валидация, UX и обработка ошибок
17 Тестирование и контроль качества
18 Пошаговое воспроизведение проекта
19 Подготовка к публикации и ограничения
20 Итоги и архитектурные выводы

ГЛАВА 01

Обзор системы и предметная область

SysAdMeow как узкоспециализированный marketplace для сетевых услуг

1.1 Идея продукта

SysAdMeow — учебное Android-приложение, объединяющее marketplace-модель и предметную область удалённой настройки сетевого программного обеспечения. Пользователь выступает либо заказчиком, либо исполнителем — sysadmin-котом. Заказчик выбирает одну разовую услугу с фиксированной стоимостью, оплачивает её, а после выполнения работы может оставить оценку и отзыв.

Концепция намеренно ограничена одной понятной бизнес-операцией: «найти специалиста → проверить компетенции → добавить услугу → оплатить → оценить». Такое ограничение делает проект пригодным для учебной демонстрации, но при этом позволяет показать полный набор типовых мобильных технологий: авторизацию, облачное хранилище, реактивное состояние, платёжный callback, транзакции, уведомления и серверные правила доступа.

На рисунке 1 изображена общая концепция SysAdMeow: два типа пользователей взаимодействуют с единым Android-приложением, которое использует Firebase для идентификации и хранения данных, beGateway для платежа и Android NotificationManager для локального уведомления.

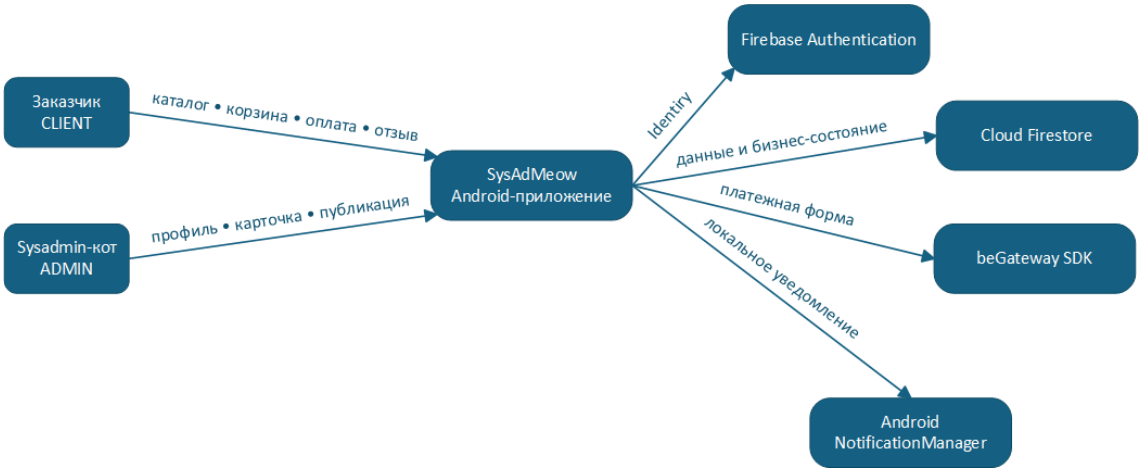


Рисунок 1 — Общая концепция системы и внешние сервисы

1.2 Почему не универсальный marketplace

В качестве концептуальных аналогов подходят универсальные биржи и маркетплейсы услуг: в них совпадают карточки исполнителей, отзывы, рейтинг и короткий сценарий заказа. Однако SysAdMeow сознательно не копирует универсальную бизнес-модель. Категории привязаны к сетевой инфраструктуре — VPN, MikroTik, Cisco, Linux и Security, — а профиль исполнителя содержит опыт и одну фиксированную стоимость услуги.

Характеристика	SysAdMeow	Обобщённый marketplace
Предметная область	Сетевое ПО и IT-support	Любые услуги
Исполнитель	Sysadmin-кот	Произвольный специалист
Цена	Фиксированная цена услуги	Может быть часовой/договорной
Категории	VPN, MikroTik, Cisco, Linux, Security	Свободное множество
Отзыв	1 отзыв пользователя на специалиста	Зависит от сервиса
Оплата	beGateway учебный сценарий	Разные платёжные модели

1.3 Границы финальной версии

- В системе присутствуют две роли: CLIENT и ADMIN.
- Публикация карточки доступна только ADMIN.
- В корзине одновременно не должно быть двух записей для одного специалиста.
- Заказ создаётся только после подтверждённого платёжного SUCCESS.
- Рейтинг является производным от отзывов и не редактируется владельцем карточки.
- Уведомление отправляется локально текущему покупателю; FCM для исполнителя в учебную версию не входит.

ГЛАВА 02

Требования, роли и пользовательские сценарии

Функциональная модель, от регистрации до отзыва

2.1 Роль заказчика

CLIENT — обычный пользователь приложения. После регистрации он получает профиль, выбирает случайную кошачью аватарку, видит опубликованный каталог и может совершить полный клиентский сценарий заказа. Данные его приватного профиля читаются только им самим, а заказ хранится за его UID.

2.2 Роль sysadmin-кота

ADMIN использует тот же Firebase-аккаунт и профиль пользователя, но дополнительно получает документ публичной карточки в коллекции sysadmins. На стороне интерфейса эта роль отображается как «sysadmin-кот». Он может создать черновик, заполнить название услуги, категории, стаж, цену и описание, а затем опубликовать карточку.

> **IMPORTANT Репутация отделена от редактора карточки**
>
> Владелец может изменять описание, цену, категории и публикацию, но не может вручную менять rating, ratingSum, reviewsCount и связанные поля последнего отзыва. Эти поля изменяются только через транзакционный сценарий добавления отзыва.

2.3 Диаграмма вариантов использования

На рисунке 2 изображены варианты использования SysAdMeow для заказчика и sysadmin-кота. Пунктирные связи показывают расширения сценариев: отзыв появляется в карточке специалиста, а публикация является специальным продолжением редактирования карточки.

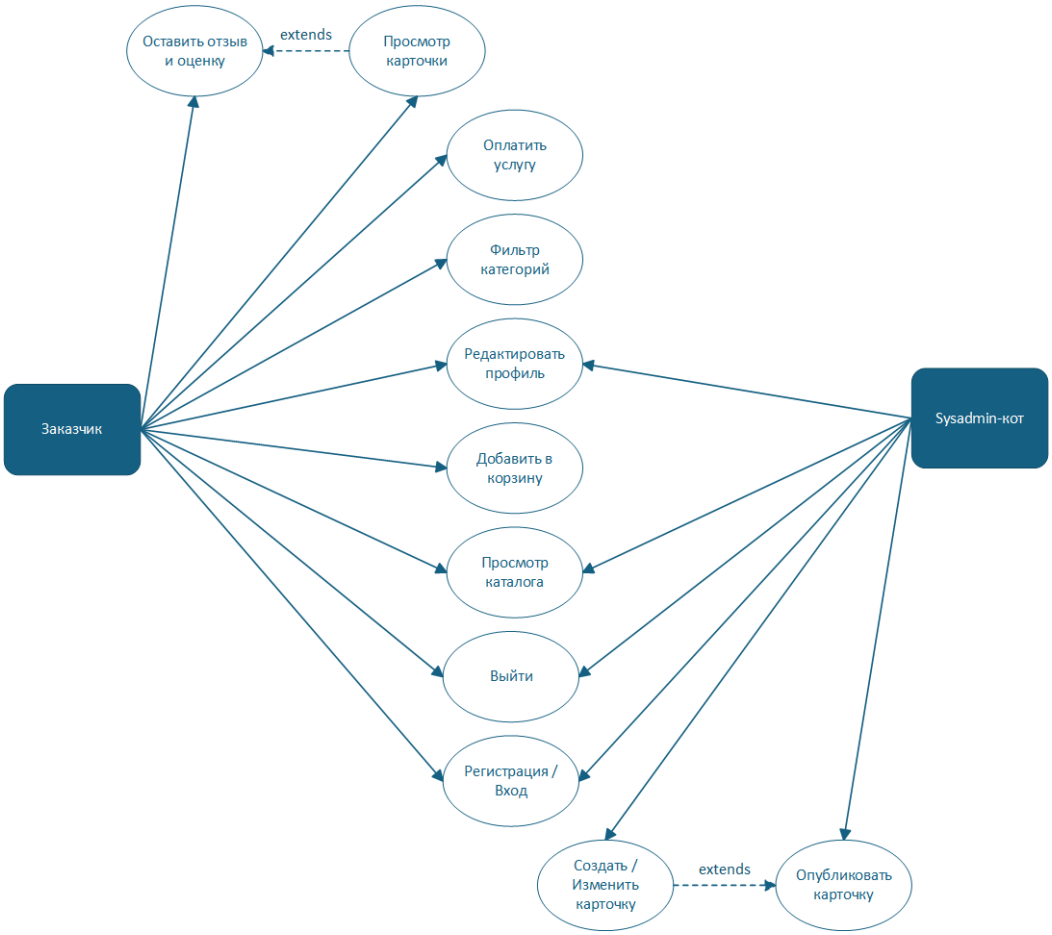


Рисунок 2 — Варианты использования SysAdMeow

2.4 Основной путь заказчика

1. Открыть приложение и пройти автоматическую проверку авторизованной сессии.
2. Зарегистрироваться или войти по Email/Password.
3. Перейти к каталогу sysadmin-котов.
4. Выбрать категорию либо оставить режим «Все».
5. Открыть карточку конкретного специалиста.
6. Проверить название услуги, компетенции, опыт, цену, рейтинг и отзывы.
7. Добавить услугу в корзину.
8. Перейти на checkout.
9. Запустить платёжную форму beGateway.
10. Получить SUCCESS, FAILED или CANCELED.
11. При SUCCESS сохранить оплаченный заказ, очистить корзину и показать итоговый экран.
12. Позднее открыть карточку и оставить оценку и отзыв, если ограничения позволяют.

2.5 Основной путь sysadmin-кота

1. Создать аккаунт с ролью ADMIN.
2. Открыть личный кабинет с автоматически созданным черновиком карточки.
3. Заполнить название услуги, категории, стаж, цену и краткое описание.
4. Сохранить карточку как черновик либо сразу опубликовать.
5. После публикации карточка становится доступной в публичном каталоге.
6. При необходимости обновлять содержание карточки без возможности вручную подменить репутационные поля.

ГЛАВА 03

Технологический стек и структура проекта

Версии, зависимости и организационные границы модулей

3.1 Финальный технологический стек

Слой	Технология	Назначение
Язык	Kotlin 2.3.0	Основной язык приложения
Android	compile/target 35, min 24	Платформенная совместимость
Gradle	AGP 8.8.2	Сборка проекта
UI	XML Views + AppCompat	Классический Android UI
Списки	RecyclerView + CardView	Каталог, корзина, отзывы
Состояние	ViewModel + StateFlow	Реактивное состояние экранов
Auth	Firebase Authentication	Email/Password и сессия
DB	Cloud Firestore	Профили, карточки, отзывы, заказы
Payment	beGateway SDK 2.6.0	Тестовый платёжный поток
Diagrams	Visio	Архитектурные диаграммы
Tests	JUnit 4	Чистая логика валидации

На рисунке 3 показана логическая структура пакетов проекта. UI-пакеты группируют экранные сценарии, data содержит репозитории и доступ к Firebase, util — валидацию и аватары, model — чистые модели данных.

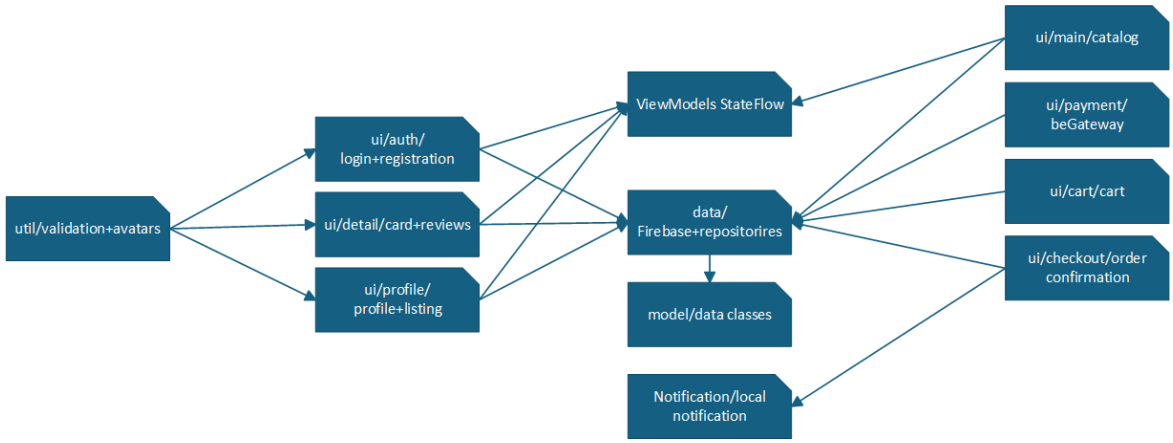


Рисунок 3 — Структура пакетов проекта

3.2 Базовая структура каталогов

```
TREE
model/              data classes
util/               Validators, CatAvatarProvider
data/               Firebase, Firestore, CartRepository
ui/auth/            LoginActivity, RegistrationActivity
ui/main/            MainActivity, catalog
ui/detail/          SpecialistDetailActivity, reviews
ui/cart/            CartActivity
ui/checkout/        CheckoutActivity
ui/payment/         PaymentActivity
ui/profile/         ProfileActivity, ProfileViewModel
notification/       OrderNotificationManager
```

Ключевая организационная граница: Activity не должна непосредственно формировать Firestore-запросы. Она передаёт действие ViewModel, а ViewModel делегирует работу Repository. Такое разделение делает UI понятным, упрощает тестирование чистой логики и изолирует инфраструктурный код.

На рисунке 4 изображена компонентная архитектура: XML Views и Activities взаимодействуют с ViewModels, ViewModels — с репозиториями, а репозитории связывают модели приложения с Firebase и корзиной. Платёжный экран дополнительно связан с beGateway SDK.

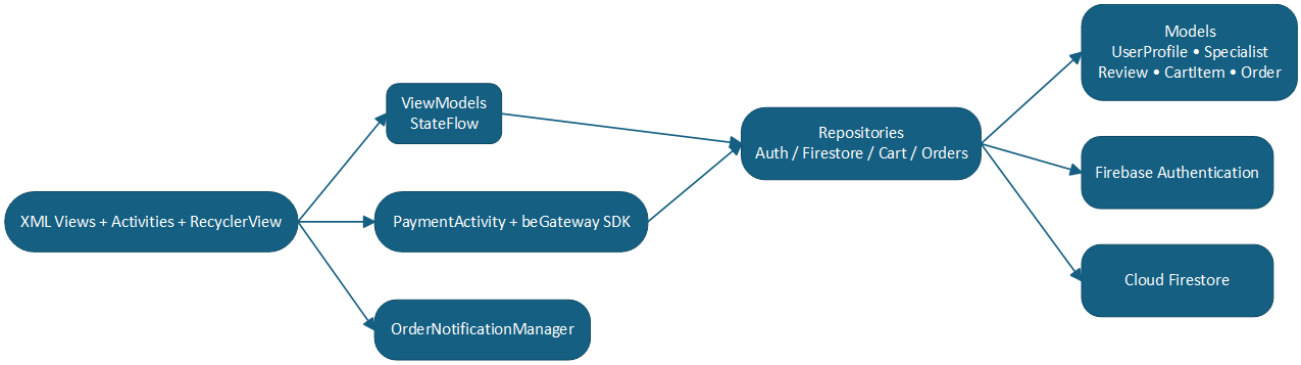


Рисунок 4 — Компонентная архитектура

ГЛАВА 04

Архитектура MVVM и управление состоянием

Как UI остаётся тонким, а состояние — наблюдаемым

4.1 Роль View

В роли View выступают Activity, XML-разметка и RecyclerView Adapter. View отвечает за отрисовку, обработку нажатия и локальную презентационную валидацию. Сложные операции — Firebase-запросы, транзакции, работа с корзиной и платежом — не должны находиться в XML-слое.

4.2 ViewModel и StateFlow

ViewModel хранит состояние, жизненный цикл которого не зависит от поворота экрана. В финальной версии ProfileViewModel, например, публикует profile, listing и message как StateFlow. Activity подписывается на них через lifecycleScope и обновляет UI при каждом изменении.

```
KOTLIN
private val  profile = MutableStateFlow<UserProfile?>(null)
val profile: StateFlow<UserProfile?> = _profile

private val _listing = MutableStateFlow<Specialist?>(null)
val listing: StateFlow<Specialist?> = _listing
```

4.3 Repository как точка доступа к данным

Repository скрывает детали инфраструктуры. AuthRepository отвечает за регистрацию, вход и выход; AdminRepository — за карточки специалистов; ReviewRepository — за транзакцию рейтинга и пагинацию; OrderRepository — за сохранение оплаченного заказа; CartRepository — за локальное состояние корзины.

4.4 Модель предметной области

На рисунке 5 изображена ER-схема финальной модели данных. Центральными сущностями являются users, sysadmins, reviews и orders; вложенная коллекция отзывов принадлежит конкретному специалисту.

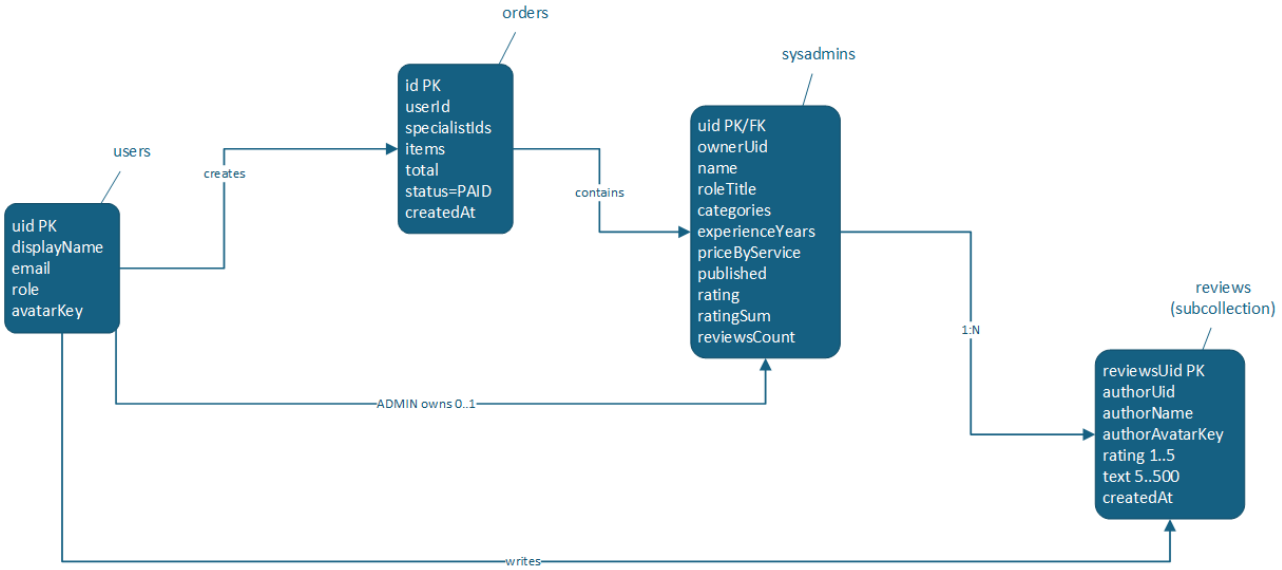


Рисунок 5 — ER-модель Cloud Firestore

На рисунке 6 изображена иерархия Firestore: пользователь связан с публичной карточкой sysadmins при роли ADMIN, карточка содержит подколлекцию reviews, а оплаченные заказы находятся в отдельной коллекции orders.

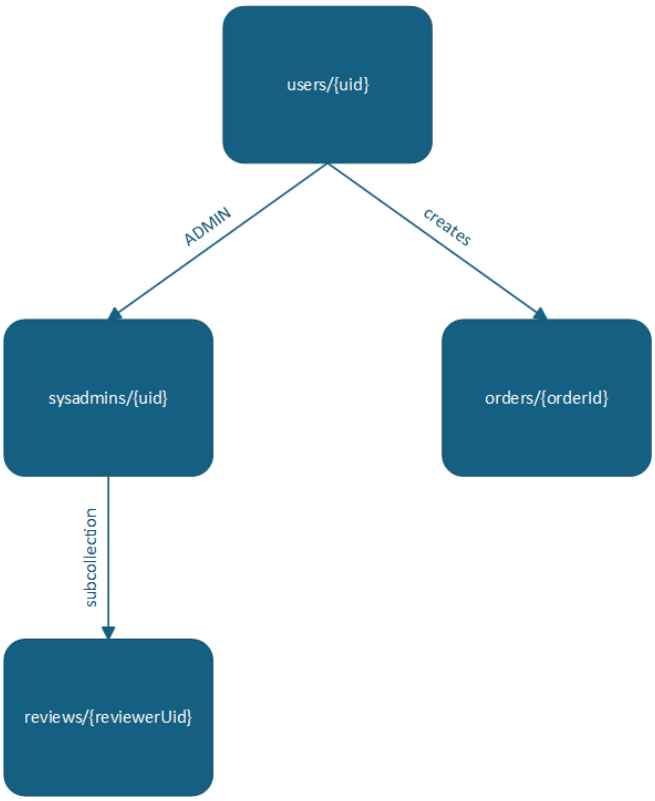


Рисунок 6 — Иерархия коллекций Firestore

ГЛАВА 05

Firebase Authentication и постоянная сессия

Идентификация пользователя как отдельная от бизнес-профиля подсистема

5.1 Принцип разделения личности и профиля

Firebase Authentication отвечает только на вопрос «кто этот пользователь?». Бизнес-профиль — имя, роль и avatarKey — хранится в Firestore. Это позволяет использовать UID Firebase как устойчивый идентификатор всех связанных сущностей и при этом отдельно ограничивать доступ к полям профиля правилами Firestore.

5.2 Регистрация

На рисунке 7 показан финальный поток регистрации: сначала выполняется локальная валидация, затем создаётся Firebase-пользователь, сохраняется документ users/{uid}, а для ADMIN создаётся черновик sysadmins/{uid}.

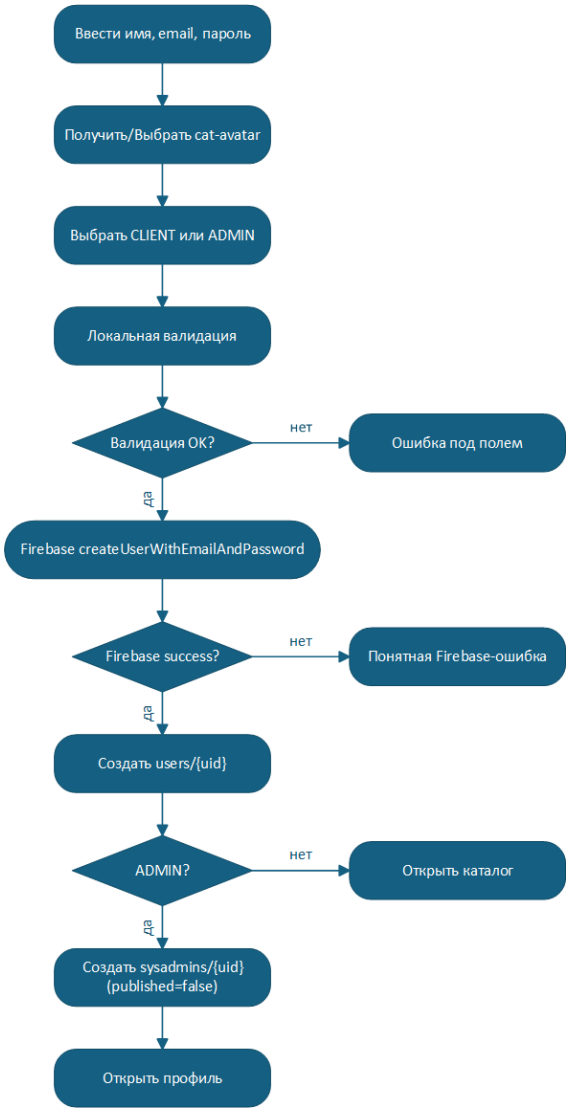


Рисунок 7 — Поток регистрации

5.3 Постоянная сессия

Firebase локально удерживает текущую аутентифицированную сессию. Поэтому LauncherActivity при каждом запуске приложения делает одну проверку currentUser. Если пользователь авторизован, приложение переходит в MainActivity; если нет — в LoginActivity. Собственного хранения паролей и отдельного SharedPreferences для авторизации не требуется.

На рисунке 8 показан стартовый роутинг: LauncherActivity проверяет FirebaseAuth.currentUser и либо открывает MainActivity, либо направляет пользователя на LoginActivity. После signOut back stack очищается.

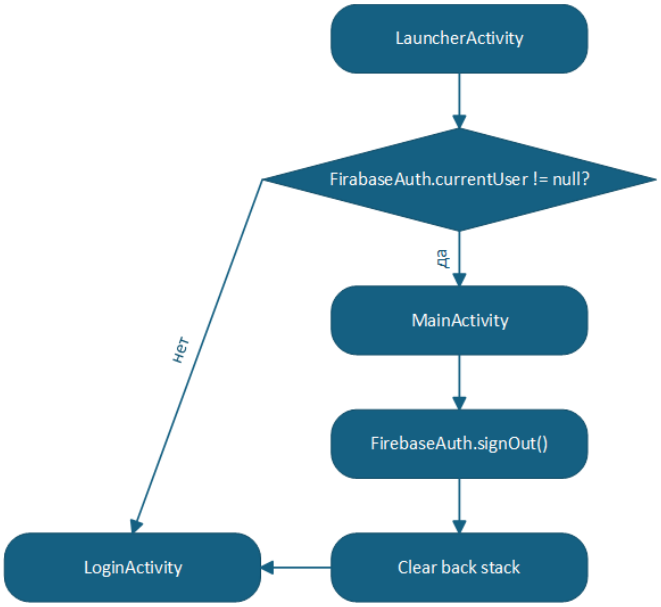


Рисунок 8 — Проверка авторизованной сессии

> **TIP Выход из аккаунта**
> Единственный штатный способ завершить сессию — FirebaseAuth.signOut(). После logout приложение очищает back stack и возвращает пользователя на экран входа.

ГЛАВА 06

Регистрация и кото-аватары

Локальная идентичность, стабильный avatarKey и UX первого запуска

6.1 Случайная аватарка

При регистрации выбирается одна из локальных vector drawable-аватарок. В Firestore сохраняется не Android resource ID, а строковый ключ avatarKey. Это архитектурно безопаснее, потому что integer resource ID не является стабильным идентификатором между сборками, тогда как строковый ключ можно однозначно сопоставить с локальным ресурсом через CatAvatarProvider.

avatarKey	Назначение
avatar_cat_ginger	Рыжий кот
avatar_cat_gray	Серый кот
avatar_cat_cream	Кремовый кот
avatar_cat_blue	Голубой кот
avatar_cat_green	Зелёный кот
avatar_cat_pink	Розовый кот

6.2 Валидация до сетевого запроса

Регистрационная форма проверяет имя (2–40 символов), базовый формат email и пароль длиной не менее 6 символов. Ошибка показывается непосредственно под полем, а ошибки Firebase — отдельным блоком. Такая модель UX сокращает число пустых сетевых запросов и сразу объясняет пользователю, что нужно исправить.

ГЛАВА 07

Cloud Firestore и модель данных

Коллекции, поля, жизненный цикл данных и связи

7.1 users/{uid}

Поле	Тип	Назначение
uid	String	Идентификатор Firebase-пользователя
displayName	String	Отображаемое имя
email	String	Email из Firebase
role	CLIENT | ADMIN	Бизнес-роль
avatarKey	String	Стабильный ключ кото-аватара

7.2 sysadmins/{uid}

Поле	Тип	Назначение
ownerUid	String	UID владельца карточки
name	String	Имя исполнителя
avatarKey	String	Аватарка
roleTitle	String	Название услуги
categories	Array<String>	Компетенции
experienceYears	Number	Стаж
priceByService	Number	Фиксированная цена
shortBio	String	Краткое описание
published	Boolean	Публичность карточки
rating	Double	Средний рейтинг
ratingSum	Number	Сумма оценок
reviewsCount	Number	Количество отзывов
latestReviewText	String	Последний отзыв
latestReviewerName	String	Автор последнего отзыва

7.3 reviews и orders

Путь	Ключевая идея
sysadmins/{uid}/reviews/{reviewerUid}	UID автора = ID документа, что естественно обеспечивает уникальность одного отзыва
orders/{orderId}	Отдельный документ с userId, массивом товаров, total, статусом PAID и createdAt

После регистрации CLIENT появляется users/{uid}. Для ADMIN дополнительно создаётся draft-карточка sysadmins/{uid}, изначально с published=false. После первого отзыва создаётся соответствующий документ в подколлекции reviews. После успешной оплаты создаётся документ orders/{generatedOrderId}.

ГЛАВА 08

Каталог, категории и главный экран

Поиск исполнителя начинается с визуально понятного каталога

8.1 Публичный каталог

MainActivity получает только опубликованные карточки специалистов. Каталог обновляется через snapshot listener, поэтому после публикации или изменения карточки изменения можно видеть без ручного refresh.

На рисунке 9 показан поток каталога: горизонтальный адаптер категорий меняет selectedCategory, ViewModel фильтрует локально полученный список, а RecyclerView отображает только подходящие опубликованные карточки.

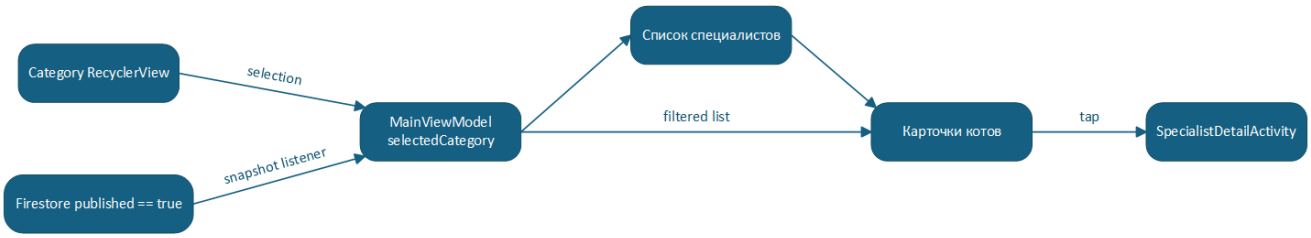


Рисунок 9 — Поток фильтрации каталога

На рисунке 10 изображён главный экран финального приложения: бренд SysAdMeow, горизонтальные фильтры категорий, карточки котов и нижняя навигация «Коты / Корзина / Профиль».

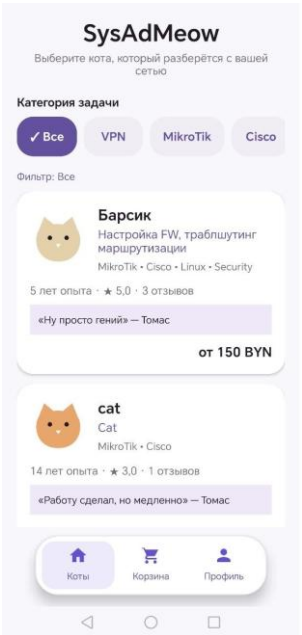


Рисунок 10 — Главный экран SysAdMeow

8.2 Состояние выбранной категории

Категория получает одновременно два визуальных признака: цветной фон и символ ✓. Это не декоративный приём, а часть доступности: состояние не зависит только от различия цветов и потому лучше считывается пользователем при нарушении цветовосприятия.

Категория	Содержимое
Все	Все опубликованные карточки
VPN	Настройка и сопровождение VPN
MikroTik	Компетенции MikroTik
Cisco	Сетевое оборудование и ПО Cisco
Linux	Linux-серверы и окружение
Security	Безопасность и защитные настройки

8.3 One-hand UX

Нижняя навигация «Коты / Корзина / Профиль» закреплена в нижней части интерфейса, а критические CTA перемещены ближе к большому пальцу. Такой подход особенно заметен на карточке специалиста и в checkout: основное действие не требует тянуться к верхней панели.

ГЛАВА 09

Карточка специалиста и отзывы

Профиль услуги как точка принятия решения

9.1 Содержимое карточки

На рисунке 11 изображена карточка sysadmin-кота: аватар, название услуги, опыт, компетенции, средний рейтинг, фиксированная стоимость, описание, форма отзыва и нижняя кнопка добавления услуги в корзину.

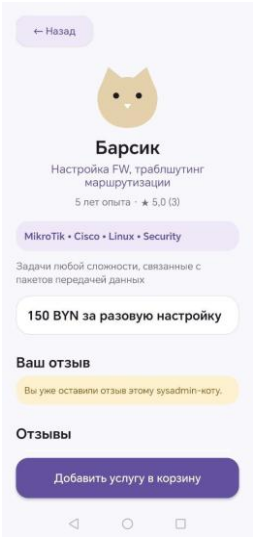


Рисунок 11 — Экран просмотра специалиста

- кошачья аватарка и имя;
- название/роль услуги;
- опыт и категории компетенций;
- средний рейтинг и число отзывов;
- краткое описание;
- фиксированная цена услуги;
- интерактивный рейтинг пользователя;
- форма текста отзыва;
- ленивое/постраничное отображение отзывов;
- CTA «Добавить услугу в корзину».

9.2 Ограничения отзыва

Правило	Реализация
Оценка 1–5	UI + Security Rules
Текст 5–500 символов	UI + Security Rules
Нельзя оценить себя	UI + request.auth.uid != sysadminId
Один отзыв на специалиста	reviewerUid используется как document ID
Нельзя редактировать/удалять	update/delete запрещены Rules

На рисунке 12 изображена последовательность транзакции отзыва: сначала читается карточка специалиста, затем проверяется отсутствие существующего review, рассчитываются новые агрегаты, создаётся review и в той же транзакции обновляется карточка.

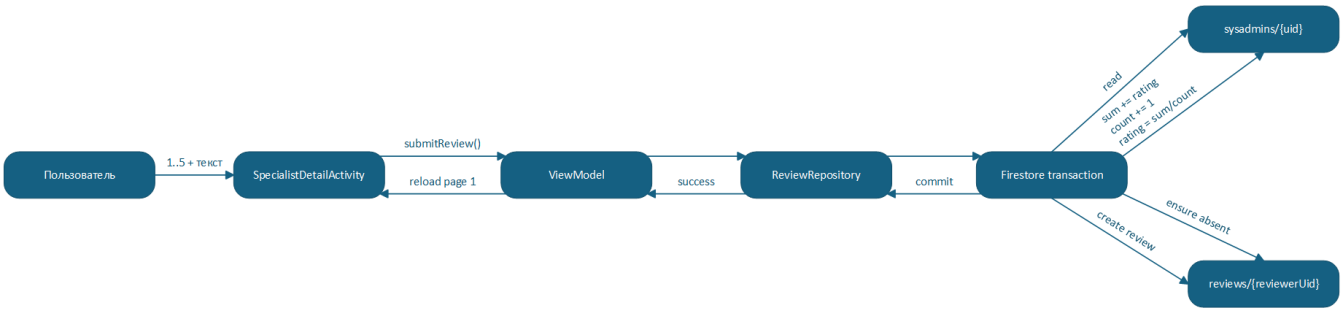


Рисунок 12 — Транзакция создания отзыва и обновления рейтинга

ГЛАВА 10

Пагинация и агрегированный рейтинг

Как система не перегружает экран отзывами и сохраняет целостность репутации

10.1 Формула рейтинга

Средний рейтинг является производным показателем: rating = ratingSum / reviewsCount. Например, после оценок 5 и 3 сумма равна 8, количество — 2, итоговый рейтинг — 4.0.

10.2 Почему нужна транзакция

Если создавать отзыв и отдельно изменять rating, между двумя операциями может возникнуть промежуточное состояние. В финальной реализации оба изменения выполняются в одной Firestore-транзакции, поэтому система логически рассматривает отзыв и изменение репутации как единую операцию.

10.3 Пагинация по пять отзывов

На рисунке 13 изображена логика пагинации: запрос сортируется по createdAt по убыванию, получает PAGE_SIZE + 1 документов, выводит первые пять, а шестой используется только для определения наличия следующей страницы.

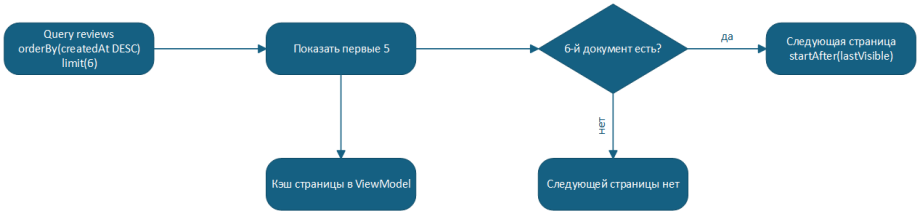


Рисунок 13 — Постраничная загрузка отзывов

```kotlin
orderBy("createdAt", Query.Direction.DESCENDING)
.limit(PAGE_SIZE + 1L)

// следующий запрос
.startAfter(lastVisibleDocument)
```

Открытые страницы кэшируются во ViewModel, поэтому возврат назад не приводит к ненужному повторному запросу. Для PAGE_SIZE = 5 шестой документ является техническим индикатором наличия следующей страницы.

ГЛАВА 11

Корзина и защита от дублей

Один специалист — одна услуга в локальной корзине

На рисунке 14 изображена корзина с одной фиксированной услугой, общей суммой и CTA перехода к оплате. В нижней части сохраняется общая навигация приложения.

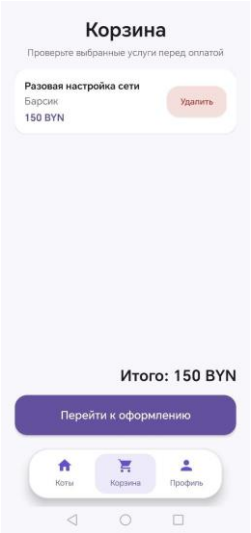


Рисунок 14 — Экран корзины

11.1 CartRepository

CartRepository хранит список CartItem в MutableStateFlow. При изменении списка CartActivity автоматически пересчитывает сумму как сумму total всех элементов.

```kotlin
if (_items.value.any { it.specialistId == item.specialistId }) {
    return false
}
_items.value = _items.value + item
return true
```

11.2 Защита от двойного открытия карточки

MainActivity использует флаг openingDetails. Он препятствует двойному запуску одной и той же SpecialistDetailActivity при двух быстрых нажатиях, а состояние сбрасывается в onResume().

> **IMPORTANT Дубли блокируются на двух уровнях**
>
> Интерфейс после добавления меняет CTA на «Уже в корзине» и блокирует повторное нажатие. Repository дополнительно проверяет specialistId. Поэтому UI-проверка не является единственной линией защиты.

ГЛАВА 12

Платёжный модуль beGateway и checkout

Учебная платёжная цепочка от token до оплаченного заказа

12.1 Архитектура платёжного потока

На рисунке 15 показано взаимодействие CheckoutActivity, PaymentActivity и beGateway SDK: checkout передаёт сумму, SDK получает token, после чего открывается card form, а результат возвращается callback-методом onPaymentFinished.

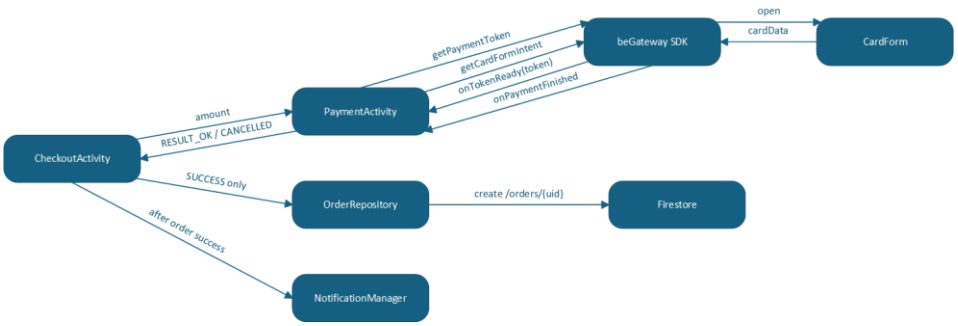


Рисунок 15 — Архитектура платёжного модуля

12.2 Финальная последовательность

1. PaymentActivity создаёт PaymentSdk через PaymentSdkBuilder.
2. Формирует TokenCheckoutData с TransactionType.PAYMENT, тестовым режимом и суммой заказа.
3. Вызывает getPaymentToken(...).
4. Получает token через onTokenReady(token).
5. Передаёт токен в checkoutWithTokenData.
6. Запускает PaymentSdk.getCardFormIntent(...).
7. Пользователь вводит данные карты в штатной форме SDK.
8. В onPaymentFinished проверяется строго response.status.name == "SUCCESS".
9. Только при SUCCESS PaymentActivity возвращает RESULT_OK в CheckoutActivity.

```kotlin
KOTLIN
private suspend fun startPayment(amountInCents: Int) {
    paymentSdk.getPaymentToken(
        TokenCheckoutData(
            Checkout(
                test = true,
                transactionType = TransactionType.PAYMENT,
                order = Order(
                    amount = amountInCents.toLong(),
                    currency = "BYN",
                    description = "SysAdMeow service",
                    trackingId = "merchant_id"
                ),
                settings = Settings(
                    saveCardPolicy = SaveCardPolicy(false)
                )
            )
        )
    )
}
```

12.3 Состояния SUCCESS / FAILED / CANCELED

На рисунке 16 изображена машина состояний оплаты: SUCCESS приводит к созданию заказа, очистке корзины и успешному состоянию; FAILED/CANCELED сохраняют корзину и оставляют возможность повторить попытку.

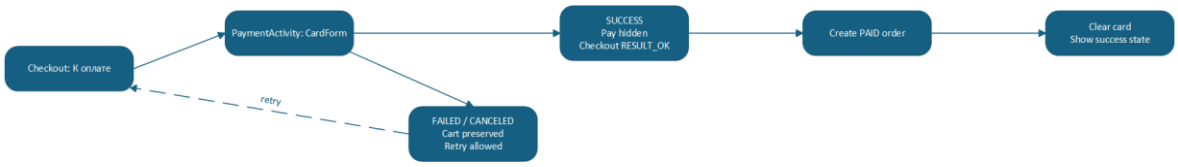


Рисунок 16 — Состояния платёжного сценария

> **CAUTION Учебный платёжный режим**
> Финальная учебная версия использует test = true и схему token → card form → callback. Для production секретные merchant-операции и получение платёжного токена следует выполнять на доверенном backend.

12.4 Checkout и заказ

CheckoutActivity не считает сам платёж успешным только потому, что пользователь вернулся с card form. Он получает RESULT_OK от PaymentActivity, после чего OrderRepository создаёт документ orders/{generatedOrderId} со статусом PAID. Только после успешного сохранения заказа очищается CartRepository.

Такое разделение важно: успешная авторизация платежа и успешное сохранение локально-облачной бизнес-сущности заказа — два разных шага. Пользователь не должен увидеть «заказ оформлен», если запись в Firestore не произошла.

ГЛАВА 13

Личный кабинет и публикация карточки

Профиль пользователя и редактор публичной карточки в одном экране

На рисунке 17 изображён личный кабинет sysadmin-кота. Общая часть показывает аватар, email, роль и имя, а для ADMIN ниже появляется редактор публичной карточки услуги: название, категории, стаж, цена, описание и кнопки сохранения.

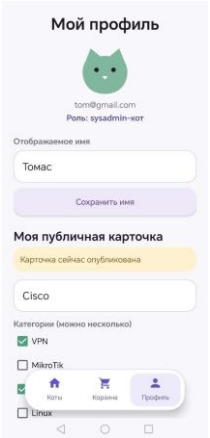


Рисунок 17 — Личный кабинет и редактор карточки

13.1 Общее содержимое профиля

- аватар;
- email;
- роль;
- редактируемое отображаемое имя;
- кнопка выхода.

13.2 Поля публичной карточки

- название услуги;
- категории компетенций;
- стаж;
- цена;
- краткое описание;
- сохранение как черновик;
- публикация.

13.3 Управление клавиатурой

При входе в профиль корневой layout получает фокус, а windowSoftInputMode настроен на скрытие клавиатуры при запуске экрана. Это защищает форму от ситуации, когда системная клавиатура сразу закрывает значительную часть интерфейса.

> **TIP Нулевые значения и hint**
>
> Для новых карточек experienceYears и priceByService могут иметь значение 0 на уровне модели. При показе формы такие нулевые значения должны преобразовываться в пустую строку, чтобы пользователь видел hint и воспринимал поле как незаполненное. При сохранении пустое поле проходит через toLongOrNull() и проверяется Validators.

ГЛАВА 14

Системные уведомления

Финальная точка сценария заказа на устройстве пользователя

На рисунке 17 изображён поток уведомления после успешного платежа и записи заказа: создаётся notification channel, проверяется разрешение Android 13+, формируется локальное уведомление и одновременно показывается success state экрана.

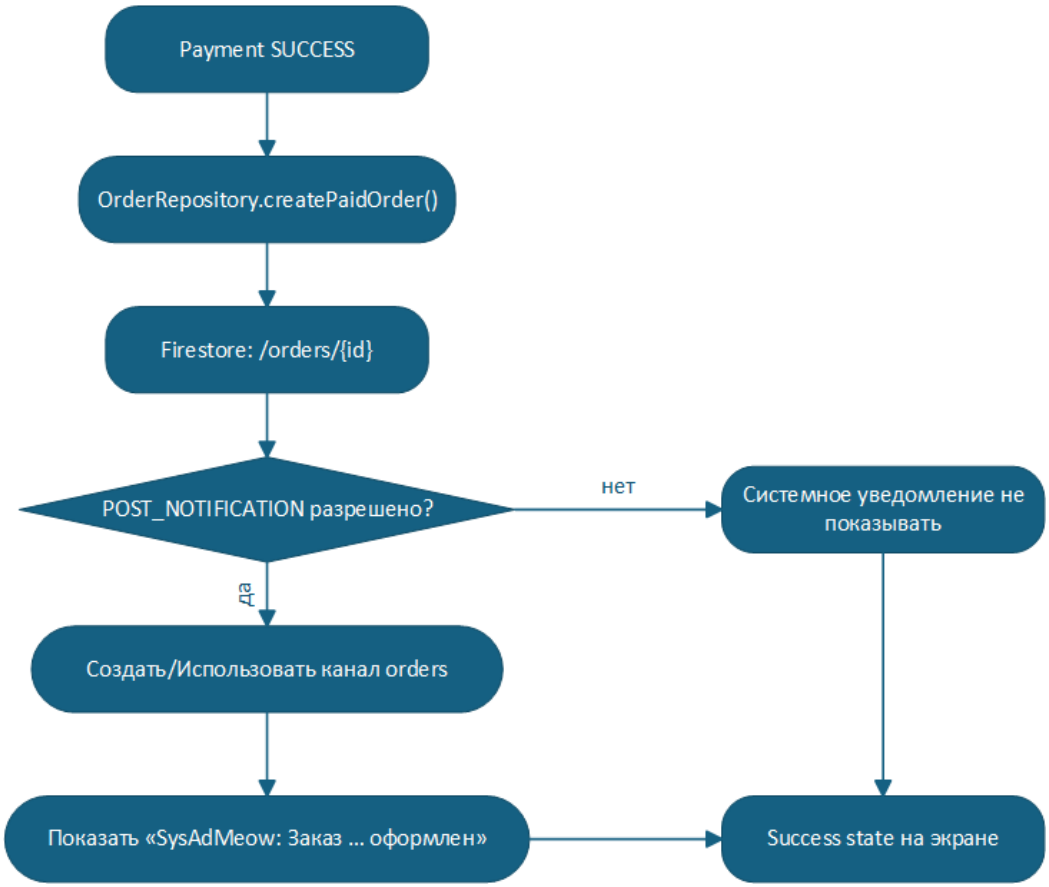


Рисунок 17 — Поток локального уведомления

14.1 Почему это локальное уведомление

OrderNotificationManager показывает уведомление непосредственно на устройстве текущего покупателя. Это не FCM push исполнителю. Такое решение соответствует границам учебной версии и позволяет продемонстрировать Android NotificationManager, не добавляя отдельный backend для push-доставки.

14.2 Android 13+

Перед отображением уведомления проверяется разрешение POST_NOTIFICATIONS. Если разрешение отсутствует, заказ всё равно считается успешно созданным — пользователь просто не получает системный toast-подобный канал уведомлений. Основной success state остаётся на экране checkout.

ГЛАВА 15

Firestore Security Rules

Серверная граница доверия: бизнес-правила нельзя оставлять только в UI

Любая проверка на стороне Android может быть обойдена модифицированным клиентом. Поэтому запреты «не менять роль», «не оценивать себя», «не менять рейтинг» и «создавать заказ только от своего UID» повторяются в Firestore Security Rules. Правила рассматриваются как часть бизнес-логики и безопасности финальной системы.

На рисунке 18 сведены основные ограничения Firestore Security Rules: приватность users, неизменяемость роли и идентичности, ограничения для sysadmins, защита от самооценки и повторных отзывов, а также привязка заказа к текущему UID.

Firestore Security Rules — основные ограничения

Ресурс	Действие	Кто	Ключевое условие
users/{uid}	read	только владелец	request.auth.uid == uid
users/{uid}	update	только владелец	role / email / uid нельзя менять
sysadmins/{uid}	read	authenticated	publicHeaders или владелец
sysadmins/{uid}	update	владелец	рейтинг и репутация неизменяемы вручную
reviews/{reviewerUid}	create	автор	1-5, текст 5-500, не самому себе
reviews/{reviewerUid}	update/delete	запрещено	запрещено
orders/{orderId}	create	authenticated	userId == auth.uid, status=PAID, total>0
orders/{orderId}	read	покупатель	resource.userId == auth.uid

Рисунок 18 — Основные Firestore Security Rules

15.1 Матрица ограничений

Ресурс	Кто может читать	Кто может изменять	Ключевая защита
users/{uid}	Только владелец	Только владелец	uid/email/role/avatar нельзя подменить
sysadmins/{uid}	Авторизованный пользователь для published или владелец	Владелец; транзакционный отзыв другого пользователя	репутацию нельзя менять вручную
reviews/{reviewerUid}	Авторизованный пользователь	Только create автором; update/delete запрещены	1 отзыв на автора + rating 1..5 + текст 5..500
orders/{orderId}	Только владелец заказа	Update/delete запрещены	userId == request.auth.uid, status == PAID, total > 0

15.2 Атомарное обновление репутации

Правило обновления sysadmins дополнительно проверяет, что изменились только репутационные поля, reviewsCount увеличился ровно на один, ratingSum вырос от одного до пяти и итоговый rating равен новой сумме, делённой на количество. Одновременно getAfter проверяет, что созданный review содержит именно ту оценку, которая была прибавлена к агрегату.

> **IMPORTANT Почему UI недостаточно**
>
> Поля роли, рейтинга и заказа являются чувствительными, потому что их изменение меняет бизнес-смысл данных. Даже идеальная Android-проверка не заменяет серверное правило, поскольку APK находится у пользователя.

ГЛАВА 16

Валидация, UX и обработка ошибок

Интерфейс должен объяснять состояние, а не заставлять пользователя угадывать

16.1 Inline-валидация

Validators.kt реализует чистую логику ошибок без зависимости от Activity. Поэтому одна и та же проверка может быть покрыта JUnit-тестами, а UI просто отображает результат рядом с конкретным полем.

Форма	Основные правила
Регистрация	Имя 2–40; корректный email; пароль ≥ 6 символов
Публичная карточка	Непустой title; хотя бы одна категория; корректные experience/price; непустое описание
Отзыв	Рейтинг 1–5; текст 5–500; нельзя оценить себя; один отзыв
Оплата	Не открывать checkout при пустой корзине; после результата блокировать повторный SUCCESS

16.2 Защита от повторных действий

Повторное нажатие — типичная мобильная ошибка. В финальной версии флаг openingDetails защищает запуск карточки, CartRepository защищает уникальность специалиста, а checkout временно отключает кнопку оплаты на время запуска платежного Activity. После FAILED/CANCELED кнопка снова активируется.

16.3 Состояния интерфейса

У экранов оплаты и заказа явно различаются промежуточное состояние, ошибка и успех. Это лучше простой модели «Toast после действия»: пользователь понимает, сохранился ли заказ, нужно ли повторять платёж и сохранилась ли корзина.

ГЛАВА 17

Тестирование и контроль качества

Проверка как отдельных функций, так и полного пользовательского пути

17.1 Unit-тесты

Чистая логика валидации находится в Validators и проверяется JUnit 4 без запуска Android UI. Базовая команда запуска — ./gradlew testDebugUnitTest.

17.2 Конечный пользовательский сценарий

На рисунке 19 изображён сквозной сценарий проверки финальной версии: регистрация ADMIN, публикация карточки, регистрация CLIENT, каталог, корзина, SUCCESS, создание заказа и отзыв с транзакционным обновлением рейтинга.



Рисунок 19 — Сквозной E2E-сценарий

1. Создать тестового ADMIN и убедиться, что создаётся users/{uid} и черновик sysadmins/{uid}.
2. Заполнить и опубликовать карточку.
3. Создать CLIENT и войти под ним.
4. Найти опубликованную карточку через каталог и фильтр.
5. Открыть карточку, убедиться в корректности цены и рейтинга.
6. Добавить услугу в корзину и убедиться, что повторное добавление блокируется.
7. Запустить тестовую оплату и проверить SUCCESS.
8. Проверить создание единственного orders-документа со статусом PAID.
9. Убедиться, что корзина очистилась и появилось локальное уведомление при разрешённом POST_NOTIFICATIONS.
10. Оставить отзыв и проверить появление review, увеличение reviewsCount/ratingSum и корректный средний rating.
11. Повторно открыть карточку под ADMIN и убедиться, что форма самооценки недоступна.

17.3 Контрольная таблица

Проверка	Ожидаемый результат
Корректный email	Проходит валидацию
Некорректный email	Блокируется до Firebase
Пароль < 6	Блокируется
Регистрация CLIENT	Создан /users/{uid}
Регистрация ADMIN	Созданы /users/{uid} и /sysadmins/{uid}
Повторный запуск	Сессия сохраняется
Logout	Открывается LoginActivity
Фильтр категории	Меняется selected state и список
Дубликат в корзине	Вторая запись не добавляется
SUCCESS оплаты	Создан PAID order
FAILED/CANCELED	Корзина сохранена
Самооценка	Блокируется UI и Rules
Повторный отзыв	Блокируется Rules
Пагинация	Не более 5 отзывов на страницу
Изменение рейтинга владельцем	Запрещено Rules

ГЛАВА 18

Пошаговое воспроизведение проекта

От пустого Android-проекта до работающего SysAdMeow

18.1 Создание Android-проекта

1. Создать Empty Views Activity в Android Studio.
2. Установить applicationId/package name: com.example.netadminmarketplace.
3. Использовать minSdk 24 и compileSdk/targetSdk 35.
4. Использовать Java 17 и Kotlin 2.3.0.
5. Использовать Android Gradle Plugin 8.8.2.

18.2 Подключение Firebase

1. Создать проект SysAdMeow в Firebase Console.
2. Добавить Android app с package com.example.netadminmarketplace.
3. Скачать google-services.json и положить его в app/.

4. Включить Authentication → Email/Password.
5. Создать Cloud Firestore.
6. Опубликовать firestore.rules из приложения.

18.3 Зависимости

```
KOTLIN
implementation("androidx.appcompat:appcompat:1.7.0")
implementation("androidx.recyclerview:recyclerview:1.4.0")
implementation("androidx.cardview:cardview:1.0.0")
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.7")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
implementation("com.google.android.material:material:1.12.0")
implementation(platform("com.google.firebase:firebase-bom:34.19.0"))
implementation("com.google.firebase:firebase-auth")
implementation("com.google.firebase:firebase-firestore")
implementation("com.github.begateway:begateway-android-sdk:2.6.0")
```

18.4 Настройка правил

Содержимое корневого firestore.rules необходимо скопировать во вкладку Firestore → Rules и нажать Publish. Без этого UI может выглядеть корректно, но операции будут получать PERMISSION_DENIED. Для воспроизведения обязательно создаются два тестовых аккаунта: ADMIN и CLIENT.

18.5 Проверка после запуска

На рисунке 20 показан итоговый стек: Android + Kotlin + Java 17, XML/View system, MVVM + StateFlow, Firebase Authentication + Firestore, beGateway SDK и документация PlantUML/JUnit.

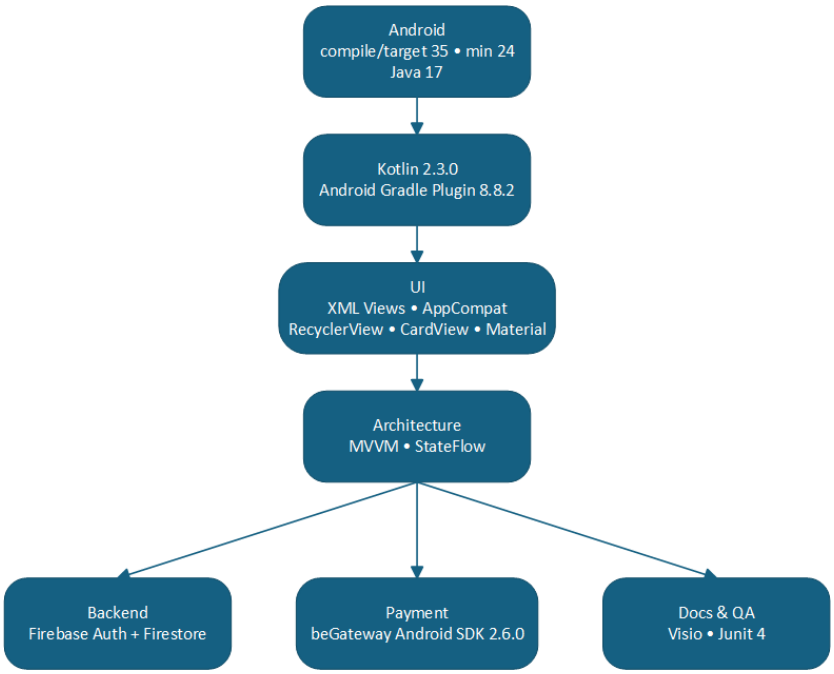


Рисунок 20 — Итоговый технологический стек

После первого запуска система должна открыть регистрацию/вход. После создания ADMIN — профиль с черновиком карточки. После публикации и входа CLIENT — каталог, карточка, корзина, checkout и полный сценарий оплаты.

ГЛАВА 19

Подготовка к публикации и ограничения

Что должно измениться до production

19.1 Production readiness

- заменить учебный merchant configuration на production;
- вынести получение payment token и другие секретные merchant-операции на доверенный backend;
- выключить payment debug/test mode;
- проверить правила через Firebase Emulator Suite и production Firebase проект;
- настроить release signing и собрать signed AAB;
- добавить Privacy Policy, crash reporting и accessibility-проверки;
- для уведомления исполнителей внедрить Cloud Functions + FCM.

> **CAUTION Локальное уведомление ≠ push-маркетплейс**
> Текущая версия уведомляет только покупателя на его устройстве. Для полноценной marketplace-модели необходимо доставлять событие исполнителю через backend и FCM.

19.2 Ограничения учебной версии

Область	Финальная учебная версия	Production направление
Платёж	test=true, SDK token flow	Backend token + production merchant
Уведомления	NotificationManager локально	FCM + Cloud Functions
Корзина	MutableStateFlow singleton	Room/DataStore по необходимости
Каталог	Firestore snapshot listener	Индексы, аналитика, кеширование по нагрузке
Security Rules	Cloud Firestore Rules	Rules + Emulator Suite + backend policies

ГЛАВА 20

Итоги и архитектурные выводы

Что именно демонстрирует SysAdMeow как учебный проект

20.1 Достигнутый результат

В финальной версии SysAdMeow реализован не отдельный экранный прототип, а связанный набор подсистем. Пользовательская история начинается с Firebase Authentication, продолжается публичным каталогом и детальной карточкой, проходит через локальную корзину и тестовую оплату, затем заканчивается созданием заказа, уведомлением и возможностью оставить отзыв.

Архитектурно система демонстрирует несколько важных принципов. Во-первых, состояние отделено от View через MVVM и StateFlow. Во-вторых, инфраструктура изолирована Repository. В-третьих, Firestore выступает не просто как хранилище, а как место, где дополнительно enforced бизнес-ограничения через Security Rules. В-четвёртых, платёж не смешивается с созданием

заказа: SUCCESS сначала приходит из SDK, а затем CheckoutActivity инициирует сохранение бизнес-сущности.

20.2 Ключевые выводы

- роль пользователя хранится отдельно от Firebase Authentication и не может быть повышена обычным update;
- репутация специалиста является вычисляемым состоянием и защищена от ручного редактирования;
- уникальность отзыва и уникальность корзины обеспечиваются не только интерфейсом, но и моделью данных/Repository;
- постраничная загрузка отзывов ограничивает объём запроса;
- оплаченный заказ появляется только после подтверждённого SUCCESS;
- документ можно использовать как основу для полного воспроизведения проекта с чистого Android-проекта.

> **NOTE Итог**
> SysAdMeow можно воспринимать одновременно как учебное приложение и как уменьшенную модель production marketplace. Главная ценность проекта — не сама тема «кот-администратор», а демонстрация связки Android UI + MVVM + Firebase + Security Rules + платёжный callback + транзакционная репутация.

ПРИЛОЖЕНИЕ A

Финальная конфигурация Gradle

Конфигурация, которую необходимо воспроизвести при сборке учебной версии

A.1 Root build.gradle.kts

```
KOTLIN
plugins {
    id("com.android.application") version "8.8.2" apply false
    id("org.jetbrains.kotlin.android") version "2.3.0" apply false
    id("com.google.gms.google-services") version "4.5.0" apply false
}
```

A.2 Kotlin compiler

```
KOTLIN
kotlin {
    compilerOptions {
        jvmTarget = org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17
    }
}
```

A.3 App dependencies

```
KOTLIN
implementation("androidx.core:core-ktx:1.15.0")
implementation("androidx.appcompat:appcompat:1.7.0")
implementation("androidx.activity:activity-ktx:1.10.0")
implementation("androidx.constraintlayout:constraintlayout:2.2.0")
implementation("androidx.recyclerview:recyclerview:1.4.0")
implementation("androidx.cardview:cardview:1.0.0")
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.7")
implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.7")
implementation("com.google.android.material:material:1.12.0")
implementation(platform("com.google.firebase:firebase-bom:34.19.0"))
implementation("com.google.firebase:firebase-auth")
```

```
implementation("com.google.firebase:firebase-firestore")
implementation("com.github.begateway:begateway-android-sdk:2.6.0")
```

A.4 Роли

```
KOTLIN
object UserRole {
    const val CLIENT = "CLIENT"
    const val ADMIN_CAT = "ADMIN"
}
```

ПРИЛОЖЕНИЕ B

Полные Firestore Security Rules

Серверные ограничения финальной учебной версии

Следующий блок является нормативной частью финальной конфигурации. Его необходимо публиковать именно как правила Cloud Firestore, а не как комментарии к Android-коду.

```
FIRESTORE.RULES
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() { return request.auth != null; }
    function myUid(uid) { return signedIn() && request.auth.uid == uid; }
    function myProfile() { return
get(/databases/$(database)/documents/users/$(request.auth.uid)); }
    function isAdminCat() { return signedIn() && myProfile().data.role == "ADMIN"; }

    match /users/{uid} {
      allow read: if myUid(uid);
      allow create: if myUid(uid)
        && request.resource.data.uid == uid
        && request.resource.data.email == request.auth.token.email
        && request.resource.data.role in ["CLIENT", "ADMIN"];
      allow update: if myUid(uid)
        && request.resource.data.uid == resource.data.uid
        && request.resource.data.email == resource.data.email
        && request.resource.data.role == resource.data.role
        && (!resource.data.keys().hasAny(["avatarKey"]) || request.resource.data.avatarKey ==
resource.data.avatarKey);
      allow delete: if false;
    }

    match /sysadmins/{sysadminId} {
      allow read: if signedIn() && (resource.data.published == true || request.auth.uid ==
sysadminId);

      allow create: if myUid(sysadminId) && isAdminCat()
        && request.resource.data.ownerUid == sysadminId
        && request.resource.data.rating == 0
        && request.resource.data.ratingSum == 0
        && request.resource.data.reviewsCount == 0;

      allow update: if myUid(sysadminId) && isAdminCat()
        && request.resource.data.ownerUid == resource.data.ownerUid
        && request.resource.data.rating == resource.data.rating
        && request.resource.data.ratingSum == resource.data.ratingSum
        && request.resource.data.reviewsCount == resource.data.reviewsCount
        && request.resource.data.latestReviewText == resource.data.latestReviewText
        && request.resource.data.latestReviewerName == resource.data.latestReviewerName;

      allow update: if signedIn() && request.auth.uid != sysadminId
        && request.resource.data.diff(resource.data).affectedKeys().hasOnly([
          "rating", "ratingSum", "reviewsCount", "latestReviewText", "latestReviewerName"
        ])
        && request.resource.data.reviewsCount == resource.data.reviewsCount + 1
        && request.resource.data.ratingSum >= resource.data.ratingSum + 1
```

```
        && request.resource.data.ratingSum <= resource.data.ratingSum + 5
        && request.resource.data.rating == request.resource.data.ratingSum /
request.resource.data.reviewsCount
        &&
!exists(/databases/$(database)/documents/sysadmins/$(sysadminId)/reviews/$(request.auth.uid))
        &&
existsAfter(/databases/$(database)/documents/sysadmins/$(sysadminId)/reviews/$(request.auth.uid
))
        &&
getAfter(/databases/$(database)/documents/sysadmins/$(sysadminId)/reviews/$(request.auth.uid)).
data.rating
             == request.resource.data.ratingSum - resource.data.ratingSum;

      allow delete: if false;

      match /reviews/{reviewerUid} {
        allow read: if signedIn();
        allow create: if signedIn()
          && reviewerUid == request.auth.uid
          && request.auth.uid != sysadminId
          && request.resource.data.authorUid == request.auth.uid
          && request.resource.data.rating is int
          && request.resource.data.rating >= 1
          && request.resource.data.rating <= 5
          && request.resource.data.text is string
          && request.resource.data.text.size() >= 5
          && request.resource.data.text.size() <= 500;
        allow update, delete: if false;
      }
    }

    match /orders/{orderId} {
      allow create: if signedIn()
        && request.resource.data.userId == request.auth.uid
        && request.resource.data.status == "PAID"
        && request.resource.data.total > 0;
      allow read: if signedIn() && resource.data.userId == request.auth.uid;
      allow update, delete: if false;
    }
  }
}
```

ПРИЛОЖЕНИЕ C

Контрольный чек-лист воспроизведения

Минимальный сценарий при проверке преподавателем или переносе проекта

- □ Android Studio открывает проект; Gradle Sync завершается без ошибок.
- □ Firebase app соответствует package/applicationId com.example.netadminmarketplace.
- □ google-services.json находится в app/.
- □ Authentication → Email/Password включён.
- □ Cloud Firestore создан, rules опубликованы.
- □ Регистрация CLIENT создаёт users/{uid}.
- □ Регистрация ADMIN создаёт users/{uid} и sysadmins/{uid}.
- □ Логин после перезапуска сохраняет сессию.
- □ Logout открывает LoginActivity и очищает back stack.
- □ ADMIN может заполнить и опубликовать карточку.
- □ CLIENT видит опубликованную карточку в каталоге.
- □ Фильтр категории меняет список и selected state.
- □ Повторное добавление специалиста блокируется.
- □ Checkout не открывается при пустой корзине.

- □ Платёж SUCCESS возвращает RESULT_OK.
- □ FAILED/CANCELED оставляет корзину.
- □ После SUCCESS создаётся один orders/{id} со статусом PAID.
- □ После сохранения заказа корзина очищается.
- □ Уведомление появляется при разрешённом POST_NOTIFICATIONS.
- □ Отзыв создаётся как reviews/{reviewerUid}.
- □ Повторный отзыв не проходит Security Rules.
- □ Самооценка не проходит Security Rules.
- □ Рейтинг считается как ratingSum/reviewsCount.
- □ Пагинация показывает максимум пять отзывов на страницу.
- □ Владелец карточки не может изменить репутацию вручную.

ПРИЛОЖЕНИЕ D

Источники проекта и навигация по исходникам

Где искать конкретную реализацию

Исходный код проекта размещён на ресурсе GitHub: https://github.com/er1argh/sysadmeow

Файл / раздел	Содержание
AuthRepository.kt	Регистрация, роль, случайная аватарка, вход/выход
LauncherActivity.kt	Сохранение сессии и стартовый роутинг
AdminRepository.kt	Карточки sysadmin-котов и каталог Firestore
ProfileViewModel.kt	Профиль и редактирование публичной карточки
ReviewRepository.kt	Транзакция оценки, агрегаты и пагинация
CartRepository.kt	Список корзины и защита от дубликатов
PaymentActivity.kt	Поток beGateway из лабораторной работы
CheckoutActivity.kt	Создание оплаченного заказа после SUCCESS

SysAdMeow

Android-маркетплейс услуг удалённой настройки сетевого ПО

Схема общей организации проекта

Sysadmin-коты настраивают сети, а вы — свое время

1. Пользователи

CLIENT
Заказчик услуг

- Просмотр каталога
- Добавление в корзину
- Оплата услуг
- Оставление отзывов
- Управление профилем

ADMIN / Sysadmin-кот
Исполнитель услуг

- Просмотр каталога
- Создание/редактирование публичной карточки
- Публикация карточки
- Управление профилем

Регистрация / Вход

Email/Password

2. Firebase Authentication

Firebase Authentication

- Email/Password
- Сохранение сессии
- Роли: CLIENT / ADMIN
- Случайная аватарка

3. Android-приложение (клиент)

Android

UI (XML + Android Views)
- Главные экраны
- RecyclerView + CardView
- Нижняя навигация (Коты / Корзина / Профиль)
- Валидация форм

MVVM + StateFlow

View → ViewModel → StateFlow

Repository слой
- Работа с Firebase
- Работа с beGateway
- Преобразование данных

Основные экраны
- Каталог (фильтры категорий)
- Карточка специалиста
- Корзина
- Checkout
- Платёж (bePaid SDK)
- Профиль
- Отзывы

4. Cloud Firestore

Cloud Firestore

Коллекции и данные
- users (пользователи)
- sysadmins (карточки специалистов)
- reviews (отзывы)
- orders (оплаченные заказы)

Firestore Security Rules
- Ограничение по ролям
- Запрет на самоназначение отзывов
- Контроль репутации
- Защита от изменения роли
- Валидация данных

5. Платёжный модуль

beGateway / bePaid SDK 2.6.0

beGateway

1. getPaymentToken() (token)
2. getCardFormIntent() (карта, NFC)
3. onPaymentFinished() (результат)

- Debug mode: true
- Store key (публичный)
- Endpoint: https://checkout.bepaid.by/ctp/api/
- TransactionType.PAYMENT
- test = true
- Валюта: BYN

6. Основной поток работы

1. **Регистрация / вход** — Проверка данных, создание записи в Firebase Auth и /users/{uid}
2. **Просмотр каталога** — Загрузка карточек специалистов из Firestore
3. **Добавление в корзину** — Проверка на дублирование специалиста
4. **Оплата** — beGateway SDK (токен → форма → результат)
5. **Создание заказа** — Запись в /orders (статус PAID)
6. **Отзыв и оценка** — Транзакция Firestore: создание отзыва + обновление рейтинга
7. **Профиль** — Просмотр и редактирование данных, для admin — управление карточкой

7. База данных (структура)

users
uid (PK)
displayName
email
role
avatarKey

orders
id (PK)
userId
specialistIds
items
total
status
createdAt

sysadmins
uid (PK/FK)
name
avatarKey
roleTitle
categories
experienceYears
priceByService
published
rating
ratingSum
reviewsCount
latestReviewText

reviews
reviewerUid (PK)
authorName
authorAvatarKey
rating
text
createdAt

8. Архитектура и технологии

- **Presentation Layer** — XML Views, Activities, RecyclerView, CardView
- **ViewModel Layer** — ViewModel, StateFlow
- **Repository Layer** — Firebase, beGateway, Models
- **Backend Services** — Firebase Authentication, Cloud Firestore
- **Payment Gateway** — beGateway / bePaid SDK 2.6.0

Технологии
- Kotlin
- Android Views
- Firebase
- beGateway / bePaid
- MVVM
- StateFlow

9. Логика ключевых процессов

Регистрация (последовательность)
- Ввод данных + валидация
- Firebase createUser()
- Создание /users/{uid}
- Для ADMIN → /sysadmins/{uid}
- Открытие нужного экрана

Платёж (SUCCESS)
- Получен токен
- Форма карты (NFC)
- onPaymentFinished()
- Создание заказа в Firestore
- Уведомление (при разрешении)
- Скрытие кнопки оплаты

Отзыв (транзакция)
- Проверка правил
- Создание /reviews/{uid}
- Обновление рейтинга
- Обновление полей карточки
- Загрузка обновлённой страницы

10. Ограничения и правила

Можно
- Просматривать публичные карточки
- Оставлять только один отзыв
- Редактировать свой профиль
- Управлять своей карточкой (ADMIN)

Нельзя
- Оставлять отзыв самому себе
- Менять роль пользователя
- Изменять рейтинг вручную
- Удалять данные

11. Легенда

- Поток данных
- Взаимодействие / вызов
- Компонент приложения
- Сервис (Firebase)
- Внешний сервис (bePaid)
- База данных
- Защита / правила
- Связь в БД

12. Жизненный цикл заказа

Выбор специалиста → Добавление в корзину → Оплата (beGateway) → Создание заказа (Firestore) → Уведомление → Выполнение услуги → Оставление отзыва → Обновление рейтинга

Изм	Лист	N докум.	Подпись	Дата	SYSADMEOW	Лит.	Масса	Масшт.
Разработал		Фёдоров В.В.			Общая схема организации проекта «SysAdMeow»			
Руководит.						Лист 1	Листов	
Консульт.								
Н.Контроп.								
Рецензент								
Утв.								